

还有，正文八个技术板块基本已经结束了。最后最值得补的是一章 **“综合对比 + 考场答题模板”**，这个比单独再讲新知识更适合开卷翻阅。它的作用是：考场上如果题目不是单点问概念，而是让你比较、设计、写伪代码，就直接翻这一章。📌

第9章 综合速查 | 开卷答题模板版

本章一句话：前面八章分别讲存储、批处理、随机读写、实时流、内存计算、嵌套列存、可靠流处理和NoSQL理论；考试中最容易混考的是“系统对比”和“给场景选方案”。

常考题型：HDFS vs HBase、MR vs Spark、Storm vs Flink、Spark vs Flink、ACID vs BASE、CAP取舍、MapReduce伪代码、HBase表设计、Storm/Flink流处理链路、Dremel r/d计算。

1. 大数据系统总图

可以先用这张总图定位题目：

存储层：

HDFS：大文件、顺序读写、批处理存储

HBase：建在HDFS上，支持RowKey随机实时读写

计算层：

MapReduce：离线批处理，两阶段Map/Reduce

Spark：内存计算、RDD、DAG、适合迭代和批处理优化

Storm：实时流处理，Topology常驻运行，低延迟

Flink：原生流处理，有状态、Watermark、Checkpoint

查询/分析：

Dremel：嵌套数据的列式交互式分析，靠r/d恢复结构

理论基础：

NoSQL：数据模型、CAP、BASE、最终一致性

一句话串起来：**HDFS** 负责把大文件可靠存下来，**MapReduce** 负责离线批量计算，**HBase** 解决随机实时读写，**Spark** 优化MR的性能和迭代计算，**Storm/Flink** 处理实时流，**Dremel** 处理嵌套数据的列式交互查询，**NoSQL** 解释这些系统背后的数据模型和一致性取舍。

2. HDFS vs HBase

对比点	HDFS	HBase
本质	分布式文件系统	分布式列族数据库
数据单位	文件 / Block	Table / Row / Column Family / Cell
访问方式	大文件顺序读写	Put / Get / Scan
适合场景	离线批处理、大文件存储	随机实时读写、海量稀疏表

对比点	HDFS	HBase
更新能力	不支持随机修改，只适合追加	支持按RowKey写入/更新
关键组件	NameNode、DataNode	Master、RegionServer、ZooKeeper
底层关系	被HBase使用	构建在HDFS上

标准答题句： HDFS 是底层分布式文件系统，适合大文件顺序读写和离线批处理；HBase 是构建在 HDFS 上的分布式列族存储系统，支持基于 RowKey 的随机实时读写。HBase 通过 HDFS 获得可靠存储能力，同时提供更接近数据库的访问接口。

3. MapReduce vs Spark

对比点	MapReduce	Spark
核心模型	Map + Reduce	RDD + DAG
中间结果	经常写磁盘/HDFS	尽量在内存中传递
执行结构	两阶段为主	DAG多阶段优化
迭代计算	每轮读写HDFS，慢	可Cache复用RDD，快
容错	Task失败重跑	RDD lineage重算
易用性	Java代码较重	API丰富，代码短
适合	离线批处理	批处理、迭代、交互分析

标准答题句： Spark 相比 MapReduce 的核心优势是内存计算、RDD Cache、DAG执行和 lineage 容错。MapReduce 多轮作业之间通常通过 HDFS 交换中间结果，磁盘 I/O 和调度开销较大；Spark 可以把多个转换操作组织成 DAG，并在内存中复用中间数据，因此更适合迭代计算和复杂数据处理。

4. Storm vs Flink

对比点	Storm	Flink
定位	实时流式计算平台	原生有状态流处理引擎
任务形态	Topology常驻运行	Job/Dataflow持续运行
延迟	低延迟	低延迟
状态支持	早期较弱，常需用户维护	Managed State，系统管理
时间语义	Event Time支持弱	原生Event Time + Watermark
窗口	早期弱，后有Windowed Bolt	原生灵活窗口
容错	Ack机制，至少一次为主	Checkpoint，可exactly-once
适合	简单实时流、低延迟处理	复杂有状态流、乱序流、可靠实时计算

标准答题句：Storm 适合低延迟实时流处理，但状态、窗口、事件时间和 exactly-once 支持较弱；Flink 是原生流处理引擎，提供事件时间、水印、托管状态、一致性检查点和保存点机制，更适合复杂有状态流处理。

5. Spark vs Flink

对比点	Spark	Flink
本质	批处理引擎	流处理引擎
流处理方式	Spark Streaming 微批	原生逐条流处理
批流一体思路	把流切成一系列小批	把批看成有限流
延迟	微批延迟	更低延迟
状态/时间	支持但不是最原生	原生状态、Watermark
适合	批处理、迭代计算、SQL分析	实时有状态流、事件时间处理

标准答题句：Spark 本质是批处理引擎，Spark Streaming 通过微批实现流处理；Flink 本质是流处理引擎，把批处理看成有限数据流。Flink 在事件时间、水印、有状态计算和 exactly-once 容错方面支持更原生，适合复杂实时流处理。

6. ACID vs BASE

对比点	ACID	BASE
核心目标	强事务、强一致	高可用、高扩展
一致性	事务一致性/强一致	最终一致性
可用性	故障时可能拒绝请求	尽量保持核心服务可用
状态	严格一致状态	允许软状态
适合	金融、订单、库存	社交、缓存、Feed、日志
代表	传统关系数据库	许多NoSQL系统

标准答题句：ACID 强调事务的原子性、一致性、隔离性和持久性，适合强一致事务场景；BASE 强调基本可用、软状态和最终一致性，适合高并发、高可用、可扩展的分布式系统。NoSQL 常采用 BASE 思想，在可容忍短暂不一致的前提下换取更高可用性。

7. CAP 考场模板

CAP三要素：

C: Consistency, 一致性。写成功后，后续读能读到最新值。
A: Availability, 可用性。每个请求都能得到响应。
P: Partition Tolerance, 分区容忍性。网络分区时系统仍能继续工作。

核心结论： 分布式系统中 **P**通常不能放弃，所以主要是在 **C** 和 **A** 之间取舍。

CP： 保证一致性，分区时可能拒绝服务。 **AP：** 保证可用性，分区时允许暂时不一致，之后最终一致。 **CA：** 在真实分布式网络分区场景下通常不现实。

标准答题句： CAP 理论指出，分布式系统无法同时完全满足一致性、可用性和分区容忍性。由于网络分区在分布式系统中不可避免，P 通常必须满足，因此系统常在 C 和 A 之间取舍。许多 NoSQL 系统偏 AP，牺牲强一致性，通过最终一致性换取高可用。

8. MapReduce 伪代码题模板

所有 MR 题都按这个五步写：

1. 输入是什么？
2. 输出是什么？
3. 最终按什么聚合？这个就是中间key。
4. Map输出什么 <k2, v2>？
5. Reduce如何处理 list(v2)？

万能模板：

```
map(k1, v1):  
    从输入中提取需要分组的key  
    emit(key, value)  
  
reduce(key, values):  
    对values进行聚合/合并/去重/拼接  
    emit(key, result)
```

WordCount：

```
map(file_id, line):  
    for word in split(line):  
        emit(word, 1)  
  
reduce(word, counts):  
    emit(word, sum(counts))
```

倒排索引：

```
map(doc_id, content):  
    for word in split(content):  
        emit(word, doc_id)
```

```
reduce(word, doc_ids):
    emit(word, unique(doc_ids))
```

反向链接图：

```
map(target, source):
    emit(target, source)

reduce(target, sources):
    emit(target, list(sources))
```

答题口诀： **Map**负责拆， **Shuffle**负责按key聚， **Reduce**负责合。

9. HBase 表设计题模板

HBase 设计题不要先想“表长什么样”，要先想访问模式。

步骤：

1. 主要查询是什么？按用户查？按时间查？按帖子查？
2. RowKey怎么设计，才能支持主要查询？
3. 是否需要倒序时间戳，支持取最近N条？
4. 哪些数据放正文表，哪些只放索引？
5. Column Family 怎么分？
6. 是否会热点？是否需要加盐/散列/反转？

Feed流固定模板：

```
user_profile:
    RowKey = user_id
    存是否大V、是否活跃

followee_list:
    RowKey = user_id
    存这个用户关注了谁，可分 normal / vip

follower_list:
    RowKey = user_id
    存这个用户有哪些粉丝

post_content:
    RowKey = post_id
    存帖子正文、作者、发布时间

user_outbox:
    RowKey = author_id
```

```
Column = reverse_timestamp#post_id

user_inbox:
  RowKey = receiver_id
  Column = reverse_timestamp#post_id
```

标准答题句： HBase 表设计应围绕访问模式进行。对于 Feed 流，应采用内容与索引分离：帖子正文存入 `post_content`，收件箱和发件箱只保存 `post_id` 索引；RowKey 采用用户 ID，列名使用倒序时间戳加 `post_id`，方便按时间倒序扫描最近内容。普通用户可写扩散，大 V 可采用读扩散或只推送活跃粉丝。

10. Storm / Flink 链路设计题模板

遇到“给一个流，设计计算链路”，先判断是 Storm 还是 Flink。

Storm 模板

```
Spout
-> ParseBolt
-> FilterBolt
-> KeyedCountBolt
-> StoreBolt
```

连接规则：

```
无状态处理：shuffleGrouping
按key统计：fieldsGrouping(key)
广播规则：allGrouping
```

例：实时 WordCount

```
SentenceSpout --shuffle--> SplitBolt
SplitBolt --fields(word)--> CountBolt
CountBolt --shuffle--> StoreBolt
```

标准答题句： Storm 拓扑设计时，Spout 作为数据源，Bolt 按处理阶段拆分。无状态解析、过滤使用 `shuffleGrouping` 进行负载均衡；需要按 key 统计或维护状态时使用 `fieldsGrouping`，保证相同 key 进入同一个 Bolt 实例。

Flink 模板

```
Source
-> map/flatMap(parse)
-> filter(valid)
```

```
-> keyBy(key)
-> window(...)
-> aggregate/process
-> Sink
```

例：每 5 分钟统计 URL PV

```
KafkaSource
-> map(parseLog)
-> filter(valid)
-> keyBy(url)
-> tumblingEventTimeWindow(5min)
-> sum(count)
-> Sink
```

标准答题句： Flink 流处理程序通常从 Source 读取数据，经 map/filter 做无状态清洗；如果需要按字段聚合或维护状态，则使用 keyBy；如果是时间范围统计，则定义窗口并配合 Watermark 处理乱序；最后通过 aggregate/process 计算结果并写入 Sink。

11. Spark Stage / Task 判断模板

Stage怎么切：

```
窄依赖不断开
遇到宽依赖 / Shuffle 就切Stage
```

常见窄依赖：

```
map、filter、flatMap
```

常见宽依赖：

```
reduceByKey、groupByKey、join、distinct
```

但注意：join 通常宽依赖，但如果两个 RDD 已经使用相同 partitioner 并共分区，可以避免 Shuffle。

Task怎么来：

一个Stage里，一个Partition通常对应一个Task
第一阶段Task数由HDFS block / HBase region决定
Shuffle后阶段Task数由shuffle partition决定

Spark WordCount Stage模板：

```
textFile -> flatMap -> map -> reduceByKey -> saveAsTextFile
```

Stage 1:

textFile + flatMap + map + shuffle write

Stage 2:

shuffle read + reduceByKey + saveAsTextFile

12. Dremel r/d 计算模板

这个一定放在开卷最显眼处。

r: repetition level, 重复层级。 看当前值是从哪个 repeated 字段开始重复。

d: definition level, 定义层级。 看字段路径上的 optional/repeated 实际存在到哪一层。

Name.Language.Code

路径：

```
Name repeated  
Language repeated  
Code required
```

所以：

```
最大 r = 2  
最大 d = 2
```

含义：

```
r=0: 新record  
r=1: 同record newName  
r=2: 同Name新Language
```


d=1: Name有, 但Language没有
d=2: Name和Language都有, Code必有

表:

value	r	d
en-us	0	2
en	2	2
NULL	1	1
en-gb	1	2
NULL	0	1

Name.Language.Country

路径:

Name repeated
Language repeated
Country optional

所以:

最大 r = 2
最大 d = 3

含义:

d=1: Name有, 但Language没有
d=2: Name和Language有, 但Country没有
d=3: Name、Language、Country都有

表:

value	r	d
us	0	3
NULL	2	2
NULL	1	1
gb	1	3
NULL	0	1

最终口诀：r 看从哪层重复，d 看定义到哪层。新 record r=0，同 record 新 Name r=1，同 Name 新 Language r=2。

13. 开卷考试答题通用结构

简答题尽量按这个结构写：

1. 先定义：它是什么。
2. 再机制：它怎么做。
3. 再作用：解决什么问题。
4. 最后局限/对比：不适合什么，和谁不同。

例：问 HDFS

HDFS 是面向大文件顺序读写的分布式文件系统。
它将文件切成多个 block，分散存储在 DataNode 上，
由 NameNode 管理文件到 block、block 到 DataNode 的元数据，
并通过多副本机制实现容错。
它适合大规模离线批处理，但不适合低延迟、小文件和随机修改。

例：问 Flink Checkpoint

Checkpoint 是 Flink 的一致性状态快照机制。
它周期性保存所有 stateful subtask 在同一逻辑输入位置上的状态，
并将状态持久化到外部存储。
故障后，应用可以从最近 checkpoint 恢复状态并继续处理。
配合 Kafka 等可重放数据源，可以实现 exactly-once。

14. 最后检查：我们已经覆盖的内容

目前正文已经完成：

第1章 HDFS
第2章 MapReduce
第3章 HBase
第4章 Storm
第5章 Spark
第6章 Dremel
第7章 Flink
第8章 NoSQL
第9章 综合对比与答题模板

按你最开始给的范围，**主干内容已经全覆盖了**。接下来最建议做的不是继续讲新章节，而是整理两份东西：

一份“考场极简索引”：一页纸告诉你看到题目翻哪章、抄哪段。**一份“预测十道简答题 + 标准答案”**：按老师可能出的题型练开卷定位和抄写。